

dev tra 5章：ゲームループ（5-02～5-05）

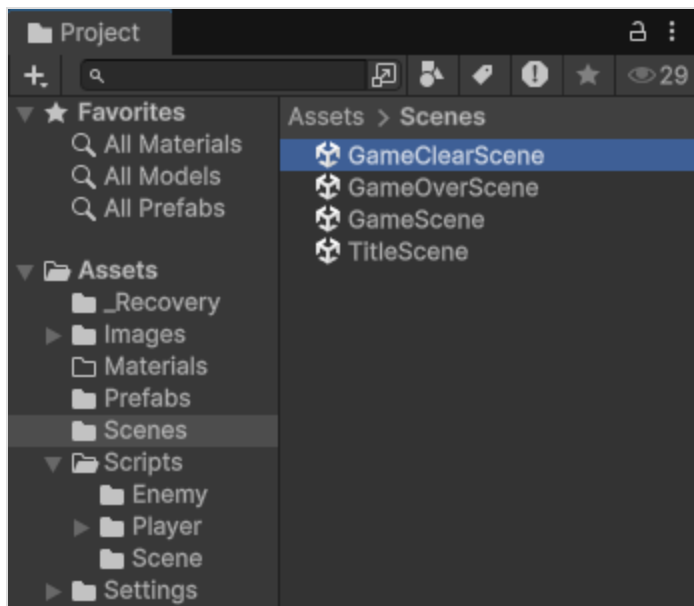
【5-02】GameClearになるようにしよう！【5章：ゲームループ】

1. クリア条件：敵を20体倒したら、ゲームクリアシーンに移行する
2. 前はゲームオーバーを作ったので、今回はその逆、ゲームクリアを作ろう。
やられたら終わり、ただだと切ないので、ちゃんと「勝ち」も用意してあげよう。
今回のクリア条件は「敵を20体倒す」です。
3. まずはゲームクリアシーンを作りましょう。
と言っても、ほぼGameOverSceneと同じものなので、ゼロから作る必要はありません。
ScenesフォルダのGameOverSceneを選択して、Ctrl + D（Macは Command + D）で複製してください。

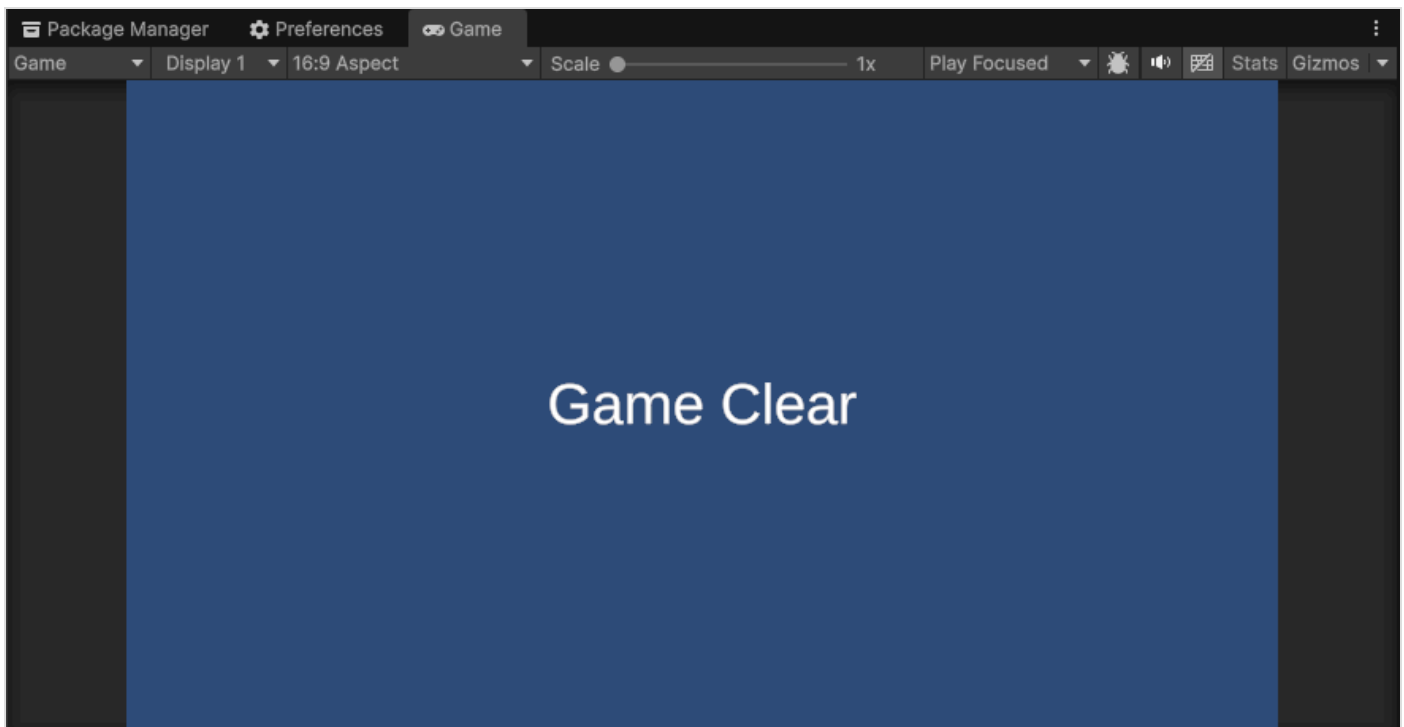
用語：複製（Duplicate）

選択中のものをコピーしてすぐ隣に作ること。ショートカットはCtrl + D。シーンでもスクリプトでもゲームオブジェクトでも使える万能技です。

4. 複製したシーンの名前を「GameClearScene」に変更しましょう。



5. GameClearSceneをダブルクリックで開いて、
 - ヒエラルキーの「GameOver」オブジェクトの名前を「GameClear」に変更
 - インспекターのテキストを「Game Over」から「Game Clear」に変更しましょう。



6. こうなっていれば、成功です！

7. では、GameSceneに戻って「敵を20体倒したらクリア」の仕組みを作っていきます。

問題です。敵を倒した数は、誰が数えるべきでしょうか？

- 敵自身？ → 倒されたら消えちゃうので数えられない
- プレイヤー？ → できなくはないけど、プレイヤーは「動く・撃つ・やられる」で手一杯

こういう「ゲーム全体のこと」は、5-01で作ったゲームの監督、GameManagerの出番です。

8. GameManager.csを開いて、このコードにしましょう。

```
using Unity.VisualScripting;
using UnityEngine;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour
{
    // シングルトン化
    public static GameManager Instance { get; private set; }

    // ゲームの状態
    public enum GameState
    {
        PLAYING,
        GAMEOVER,
        GAMECLEAR,
    }

    // ゲームの状態
    public GameState gameState_ { get; private set; }

    // プレイヤーの体力
```

```

[SerializeField] private PlayerHealth playerHealth_;

// プレイヤーのやられアニメーション
[SerializeField] private PlayerDeathAnimation playerDeathAnimation_;

// クリアに必要な撃破数
[SerializeField] private int clearTargetCount_ = 20;

// クリアしてからシーン切り替えまでの待機時間 (秒)
[SerializeField] private float clearWaitTime_ = 1.5f;

// 敵の撃破数
private int defeatCount_ = 0;

void Awake()
{
    // instance元を設定
    Instance = this;
}

// Start is called once before the first execution of Update after the MonoBehaviour is created
void Start()
{
    // nullチェック
    Debug.Assert(playerHealth_ != null, "PlayerHealth が設定されていません");
    Debug.Assert(playerDeathAnimation_ != null, "PlayerDeathAnimation が設定されていません");

    // 開始時のゲーム状態を設定
    gameState_ = GameState.PLAYING;
}

// Update is called once per frame
void Update()
{
    // 各状態の更新
    switch (gameState_)
    {
        {
            case GameState.PLAYING:
                // プレイヤーが死んだらゲームオーバー
                if (!playerHealth_.IsAlive())
                {
                    gameState_ = GameState.GAMEOVER;
                }

                // 敵を規定数倒したらゲームクリア
                if (defeatCount_ >= clearTargetCount_)
                {
                    gameState_ = GameState.GAMECLEAR;
                }

                break;
            case GameState.GAMEOVER:
                // ゲームオーバーシーンに切り替え
                if (playerDeathAnimation_.isAnimationFinished_)

```

```

        {
            ChangeGameOverScene();
        }
        break;

    case GameState.GAMECLEAR:
        // 少し待ってからゲームクリアシーンに切り替え
        clearWaitTime_ -= Time.deltaTime;
        if (clearWaitTime_ <= 0.0f)
        {
            ChangeGameClearScene();
        }
        break;

    default:
        break;
    }
}

/// <summary>
/// シーン切り替え
/// </summary>
void ChangeGameOverScene()
{
    SceneManager.LoadScene("GameOverScene");
}

/// <summary>
/// シーン切り替え（ゲームクリア）
/// </summary>
void ChangeGameClearScene()
{
    SceneManager.LoadScene("GameClearScene");
}

/// <summary>
/// 敵の撃破数を加算
/// </summary>
public void AddDefeatCount()
{
    // 撃破数を加算
    defeatCount_++;

    // わかりやすいようにログを出す
    Debug.Log("Defeat Count : " + defeatCount_);
}
}

```

GameManager.cs

9. ポイントは3つ。

- 撃破数を数える変数 defeatCount_ と、それを+1する AddDefeatCount() を追加した

- PLAYING中に「撃破数がクリア目標に届いたか」をチェックして、届いたらGAMECLEARに切り替え
- GAMECLEARになったら、少しだけ待ってからゲームクリアシーンへ移動

すぐにシーンを切り替えず1.5秒待っているのは、20体目の敵が吹っ飛ぶところを見届けてから移動したほうが気持ちいいからです。この辺はお好みで。

10. 次に、「敵が倒されたよ」とGameManagerに報告する側を作ります。

敵のHPを管理しているのは誰だったか覚えていますか？ そう、EnemyHealthです。

EnemyHealth.csをこのコードにしましょう。

```
using UnityEngine;

public class EnemyHealth : MonoBehaviour
{
    [SerializeField] private int hp_ = 3;

    private EnemyDamageFlash flash_;
    private EnemyDeathAnimation deathAnimation_;
    void Start()
    {
        flash_ = GetComponent<EnemyDamageFlash>();
        deathAnimation_ = GetComponent<EnemyDeathAnimation>();
    }

    public void Damage(int damage)
    {
        // ダメージ計算
        hp_ -= damage;

        // わかりやすいようにログを出す
        Debug.Log("Enemy HP : " + hp_);

        // 赤色にセット
        flash_.Flash();

        // hpがゼロ以下ならエネミーを削除
        if (hp_ <= 0)
        {
            // GameManagerに撃破数を報告
            GameManager.Instance.AddDefeatCount();

            deathAnimation_.StartAnimation();
        }
    }

    /// <summary>
    /// 生きているかどうか
    /// </summary>
    /// <returns></returns>
    public bool IsAlive()
    {
        return (hp_ > 0);
    }
}
```

```
}  
}
```

EnemyHealth.cs

11. 追加したのは `GameManager.Instance.AddDefeatCount();` の1行だけです。

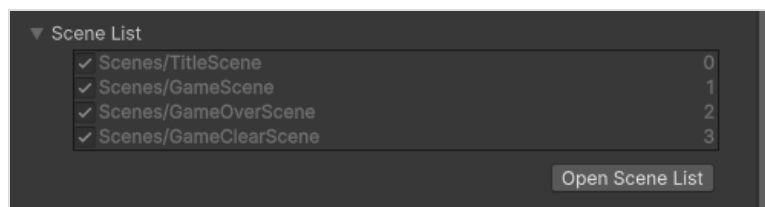
5-01でGameManagerをシングルトンにしておいたおかげで、敵は「GameManager.Instance」と書くだけで、どこからでも監督に報告できます。仕込みが効いてくると気持ちいいですね。

ちなみに、画面外に逃げられた敵は倒したことにはなりません。あくまで「撃破」した数だけカウントされます。

12. 最後に、GameClearSceneをUnityに登録します。5-01でやったやつです。

File > Build Profiles を開いて、Open Scene Listをクリック。

GameClearSceneを開いた状態で「Add Open Scenes」を押すと、GameClearSceneがリストに追加されます。

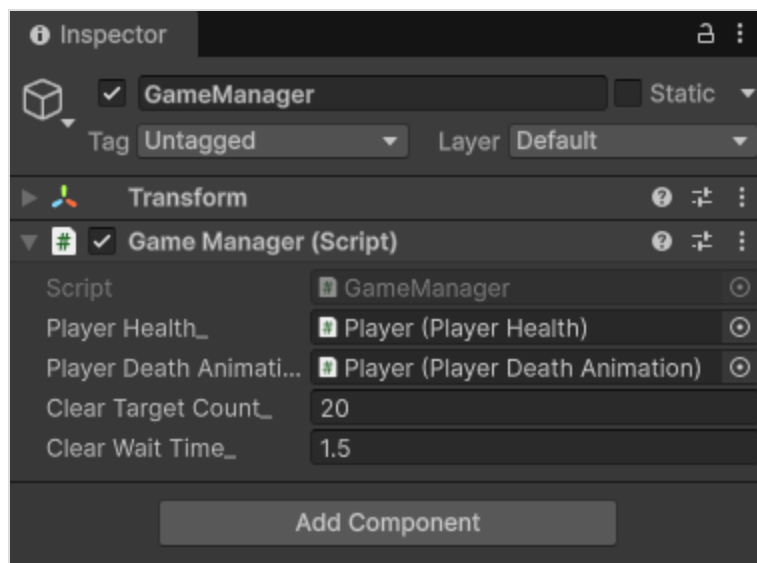


※スクショは次回のTitleSceneまで追加済みのものです。この時点ではGameClearSceneが増えていればOK！

13. では、GameSceneに移行してテストプレイ！ 敵を20体倒すと。。。ゲームクリアシーンに移動したら完成！

20体倒すのが面倒な人は、ヒエラルキーのGameManagerを選択して、インスペクターの「Clear Target Count」を2とかに変えてテストしましょう。

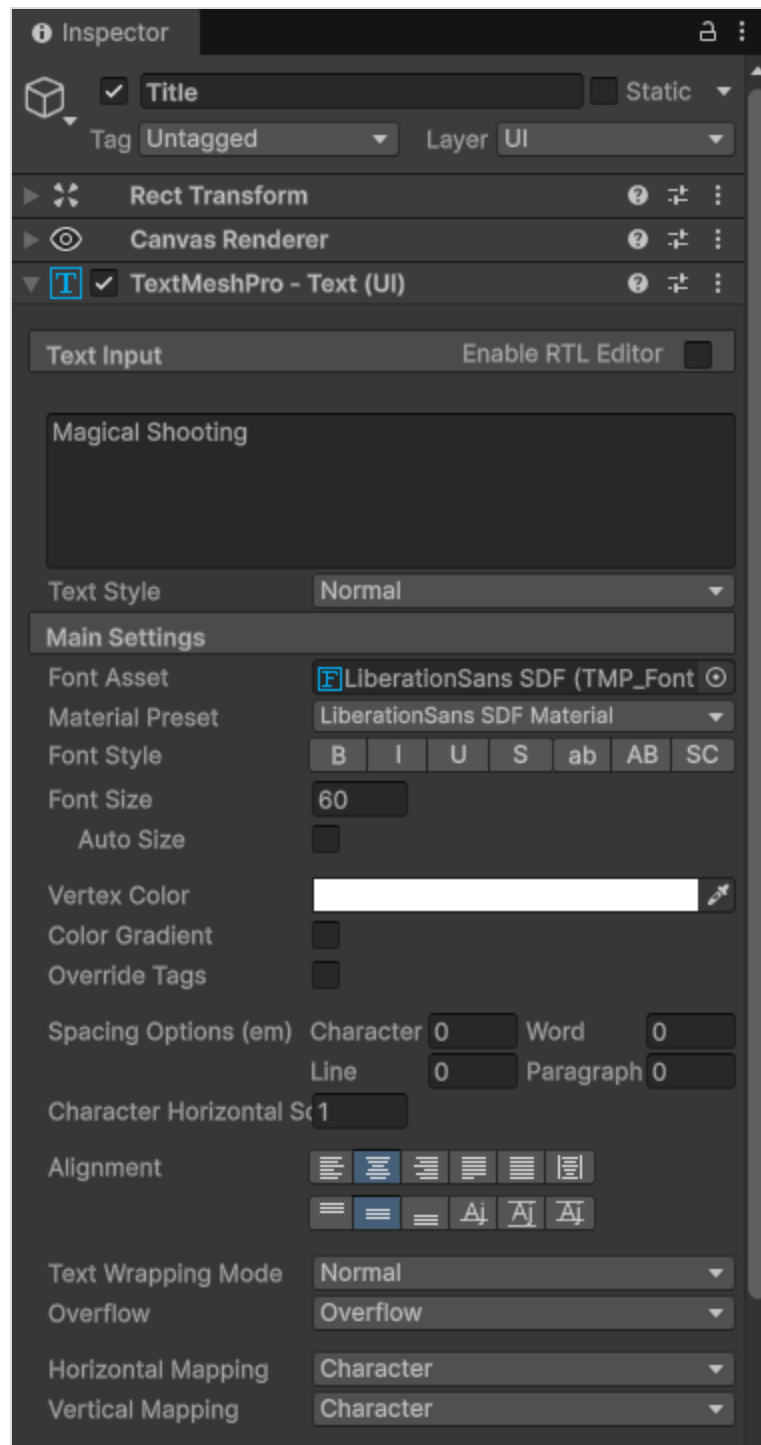
2-10でやった[SerializeField]のおかげで、コードを書き換えずに調整できます。テストが終わったら20に戻すのを忘れずに！



14. ↓ --- 質問コーナー --- ↓

【5-03】タイトルから始められるようにしよう！【5章：ゲームループ】

1. クリア条件：タイトル→ゲーム→ゲームオーバー／ゲームクリア→タイトル のループを完成させる
2. ゲームオーバーとゲームクリアができたので、いよいよ仕上げです。
世の中のゲームを思い出してみてください。起動したらまずタイトル画面が出て、遊び終わったらまたタイトルに戻ってきますよね。
この「1周まわる流れ」のことをゲームループと呼びます。これができると、一気に「ゲームらしく」なります。今回で5章のタイトル回収です。
3. まずはタイトルシーンを作りましょう。もうお馴染み、GameOverSceneをCtrl + Dで複製して、名前を「TitleScene」にしてください。
4. TitleSceneを開いて、テキストをタイトルっぽくします。
 - ヒエラルキーの「GameOver」オブジェクトの名前を「Title」に変更
 - テキストを「Magical Shooting」に変更（自分のゲーム名にしてもOK！）
 - Font Sizeを60に変更
 - 見切れる場合は、Rect TransformのWidthを600、Heightを100くらいに
 - Pos Yを60にして、少し上に配置



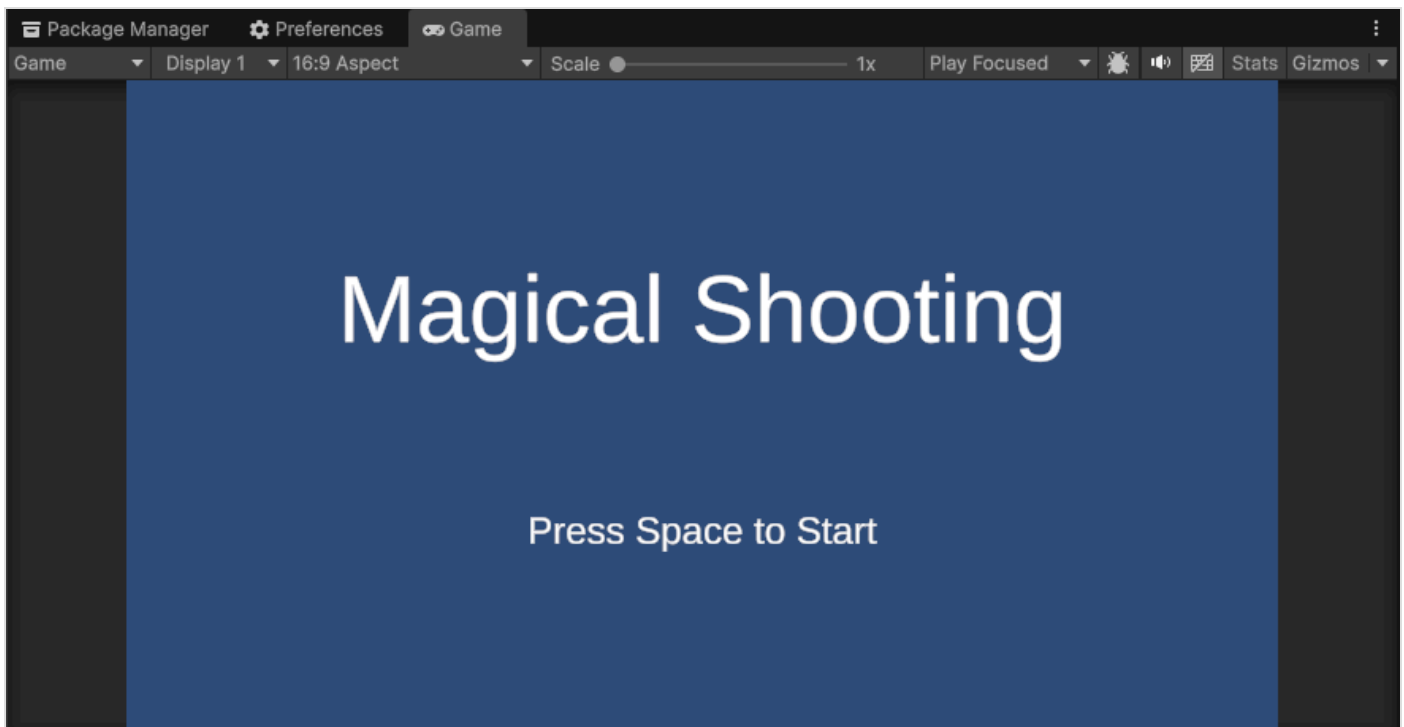
5. タイトルだけだと寂しいので、「スペースキーで始まるよ」の案内も置きましょう。

ヒエラルキーのTitleを選択してCtrl + Dで複製し、

- 名前を「PressKey」に変更
- テキストを「Press Space to Start」に変更
- Font Sizeを24に変更
- Pos Yを-80にして、タイトルの下に配置

してください。

6. このような画面になっていれば、成功です！



7. では、「キーを押したらシーンが切り替わる」スクリプトを作ります。

Scripts > Sceneフォルダに「SceneChangeOnKey」スクリプトを作成して、このコードを書きましょう。

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneChangeOnKey : MonoBehaviour
{
    // 押されたら遷移するキー
    [SerializeField] private KeyCode key_ = KeyCode.Space;

    // 遷移先のシーン名
    [SerializeField] private string nextSceneName_ = "";

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    void Start()
    {
        // nullチェック
        Debug.Assert(nextSceneName_ != "", "nextSceneName_ が設定されていません！");
    }

    // Update is called once per frame
    void Update()
    {
        // キーが押された瞬間にシーンを切り替える
        if (Input.GetKeyDown(key_))
        {
            SceneManager.LoadScene(nextSceneName_);
        }
    }
}
```

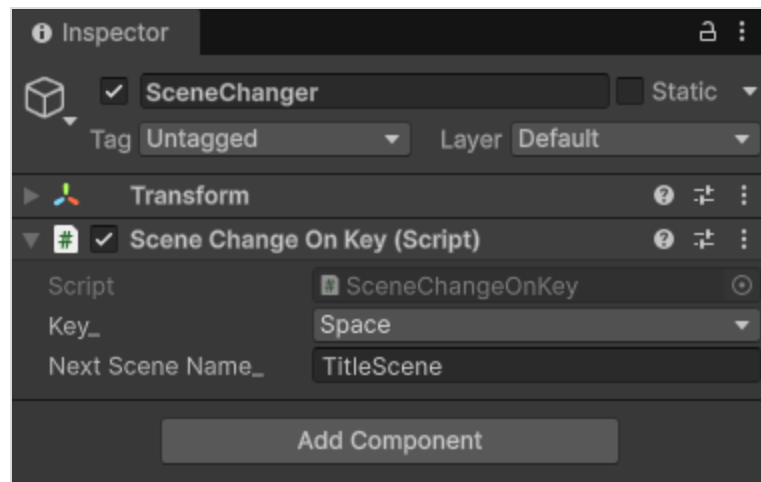
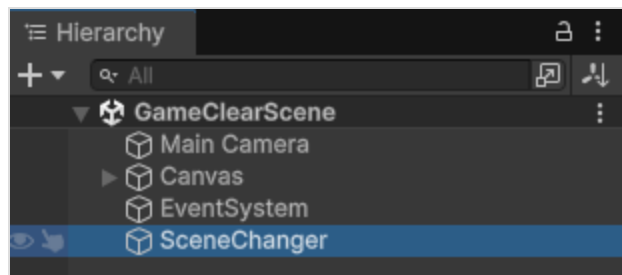
用語：KeyCode

キーボードのキー1つ1つに付いている名前。KeyCode.Spaceはスペースキー、KeyCode.Returnはエンターキー。PlayerMoveで使ったKeyCode.UpArrowの仲間です。

8. このスクリプトのえらいところは、「どのキーで」「どのシーンに」移動するかを[SerializeField]にしてあるところです。
つまりコードは1個だけ書いて、あとはインスペクターで設定を変えれば、タイトルでもゲームオーバーでもどこでも使い回せます。プログラムは書けば書くほどバグるので、使い回せるものは使い回しましょう。
9. TitleSceneのヒエラルキーで「Create Empty」をして、空のゲームオブジェクトを作成しましょう。
名前は「SceneChanger」です。
10. SceneChangerに、Add ComponentでSceneChangeOnKeyを追加して、
 - Key : Space
 - Next Scene Name : GameScene

と設定しましょう。

※シーン名は1文字でも間違えると動きません！「gamescene」や「Game Scene」はダメです。不安な人はScenesフォルダの名前をコピーしましょう。



※インスペクターのスクショはGameClearSceneに置いたもの（Next Scene NameがTitleSceneになっています）

11. 同じことを、GameOverSceneとGameClearSceneにもやります。
それぞれのシーンを開いて、SceneChangerオブジェクトを作成し、SceneChangeOnKeyを追加。ただし今度は、
 - Key : Space
 - Next Scene Name : TitleScene

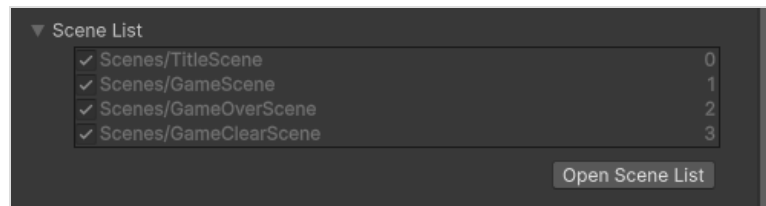
です。ゲームが終わったらタイトルに帰ってくるわけですね。

12. 最後に、TitleSceneをUnityに登録します。File > Build Profiles > Open Scene Listで、TitleSceneを開いた状態で「Add Open Scenes」。
13. そして今回はもうひと手間。追加したTitleSceneをドラッグして、Scene Listの一番上に移動してください。

これで、ゲームオーバーシーンに移行できる・・・のは前回までで出来ていましたが。

書き出した（ビルドした）ゲームは、Scene Listの一番上のシーンから起動するんです。

エディターでは今開いているシーンからプレイが始まるので気づきませんが、ここを並べ替えておかないと、完成品がいきなりゲーム本編から始まってしまいます。



14. では、TitleSceneを開いてテストプレイ！

タイトル → スペースでゲーム開始 → やられたらゲームオーバー → スペースでタイトル → もう一回遊んで今度はクリア → スペースでタイトル。

ぐるぐる回れたら、ゲームループ完成です！！

15. ちなみに「タイトルでスペースを押したら、ゲーム開始と同時に弾が出ちゃわない？」と思った人、鋭い。

GetKeyDownは「押しっぱなし」には反応せず、「押した瞬間」だけ反応するので、シーンが切り替わった後に押し直さない限り弾は出ません。安心してください。

16. ↓ --- **質問コーナー** --- ↓

【5-04】ゲームを書きだそう！【5章：ゲームループ】

1. クリア条件：自分のPC向けにゲームをビルドして、Unityがなくても遊べる状態にする
2. ゲームループが完成したので、いよいよゲームを「書き出し」ます。

用語：ビルド

Unityのプロジェクトを、Unityを持っていない人でも遊べる形に変換すること。今まではUnityエディターの中でしか動かせませんでしたが、ビルドすると単体のゲームとして動くようになります。

3. 書き出す先はWindowsアプリ、Macアプリ、スマホ、Web、いろいろ選べます。
今回はまず、一番シンプルな「自分のPC向け」に書き出します。

- Windowsの人 → exeファイル
- Macの人 → アプリ（.app）

ブラウザで遊べるWeb向けの書き出しは、次回unityroomに投稿するときにやります。

4. その前に、ビルド前のごしらえがあります。

実は、このままビルドすると赤いエラーが出て失敗する人が多いはず。犯人はスクリプトの先頭にいる、こういうやつら。

```
using UnityEditor.Analytics;  
using NUnit.Framework;
```

「UnityEditor」で始まるものは、Unityエディターの中でだけ使える機能です。ビルドしたゲームにはエディターは入っていないので、「そんなものは無い！」とエラーになります。NUnitも同じくテスト用なのでビルドには入りません。

これ、自分で書いた覚えがないと思います。VSCodeやVisual Studioが入力補完のついでに勝手に追加してくるんです。ひどい話ですね。

5. 対処は簡単で、使っていないusingを消すだけです。

自分のスクリプトを見て、「UnityEditor」「NUnit」で始まるusingがあったら削除して保存しましょう。

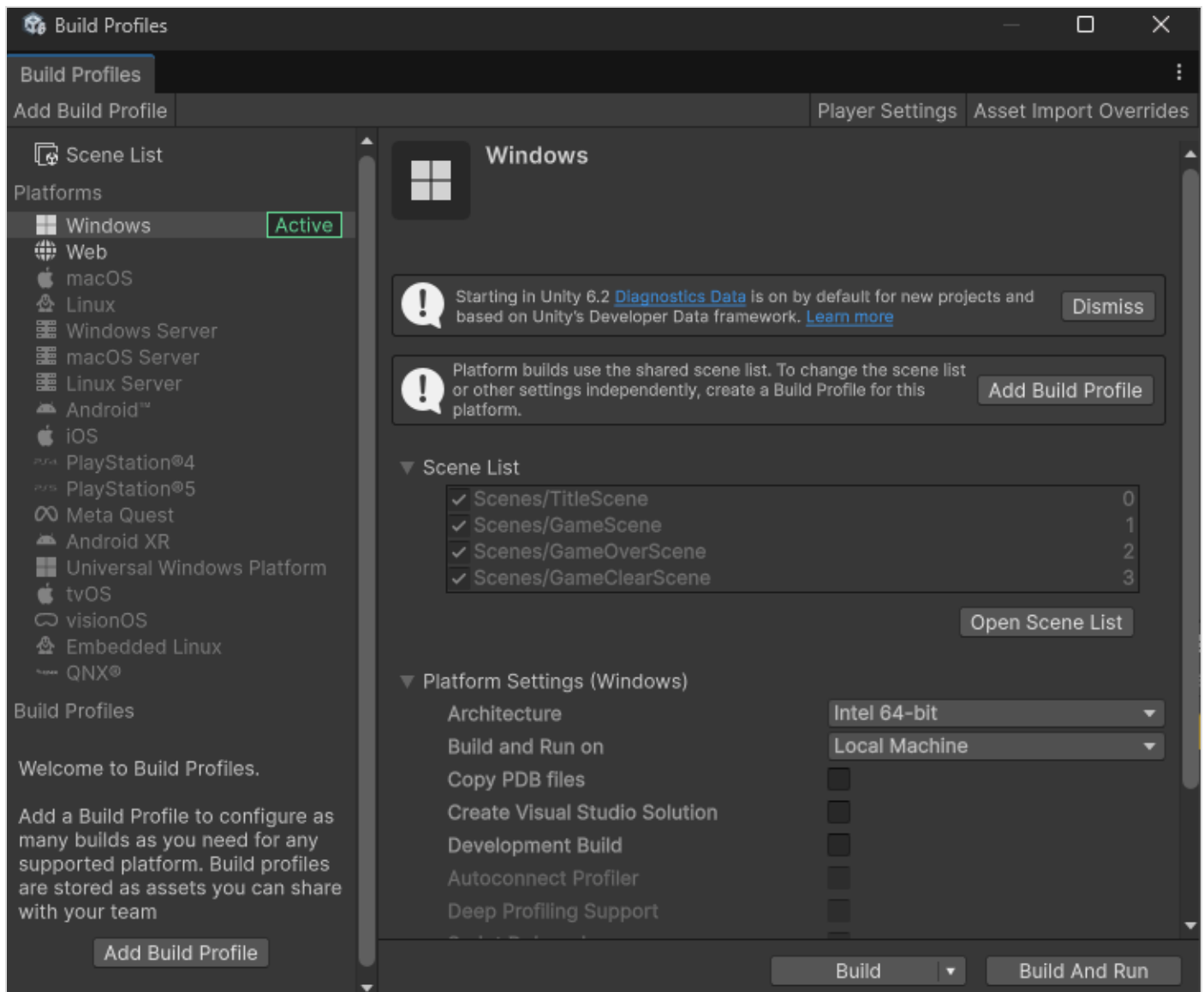
僕の環境では、PlayerMove.csに「using UnityEditor.Analytics;」、EnemyDeathAnimation.csに「using NUnit.Framework;」などが紛れ込んでいました。

消していいのかわからないusingは、試しに消してみてもConsoleにエラーが出なければ不要だったということです。エラーが出たら戻せばOK。

6. ごしらえができれば、File > Build Profiles を開きましょう。

Platformsで、Windowsの人は「Windows」に緑の「Active」が付いていることを確認してください。Macの人は「macOS」がActiveのはずです。

※Activeになっていない場合は、自分のOSを選択して「Switch Platform」を押しましょう。



ちなみに、WindowsでMac向けをビルド（またはその逆）もできなくはないのですが、署名などの面倒な問題があるので、基本は「自分のOS向け」でOKです。

7. Scene Listに4つのシーンが入っていて、TitleSceneが一番上にあることも、前回の復習がてら確認しておきましょう。

8. では、「Build」ボタンを押しましょう。

書き出し先のフォルダを聞かれるので、プロジェクトの外にフォルダを作って選択してください。

僕は「C:\Users\ユーザー名\Builds\MagicalShooting\Windows」というフォルダを作りました。

※フォルダのパスに日本語が入らない場所が安全です。

9. あとは待つだけ。初回は数分かかります。お茶でも飲みながら、完成に思いを馳せましょう。

※途中でエラーが出て止まった場合は、Consoleの赤いエラーを読んでみましょう。だいたいさっきのusingの消し忘れです。

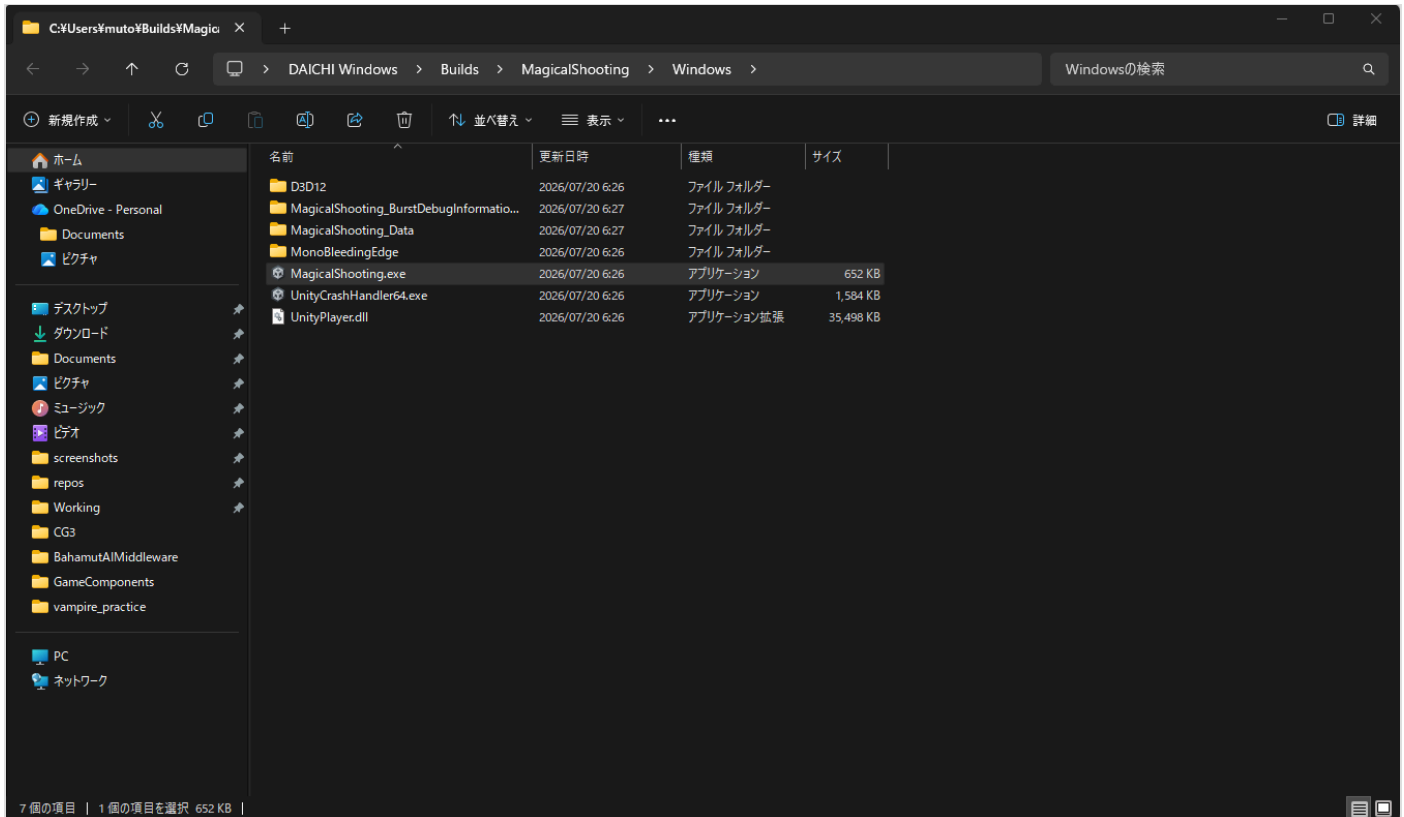
10. ビルドが終わると、フォルダの中にこんなものができています。

Windowsの人：

- MagicalShooting.exe ← これがゲーム本体！
- UnityPlayer.dll など、いくつかのファイルとフォルダ

Macの人：

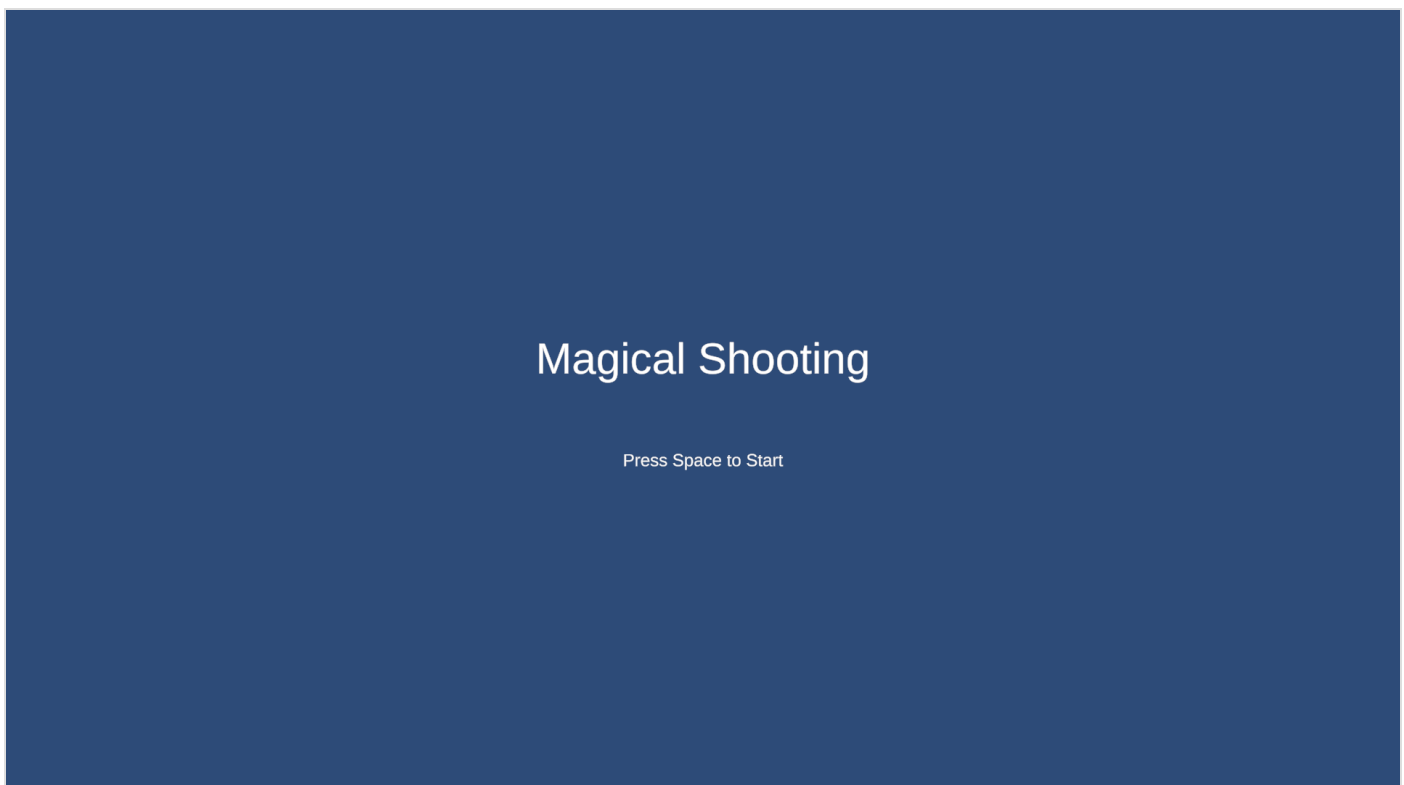
- MagicalShooting.app ← これがゲーム本体！



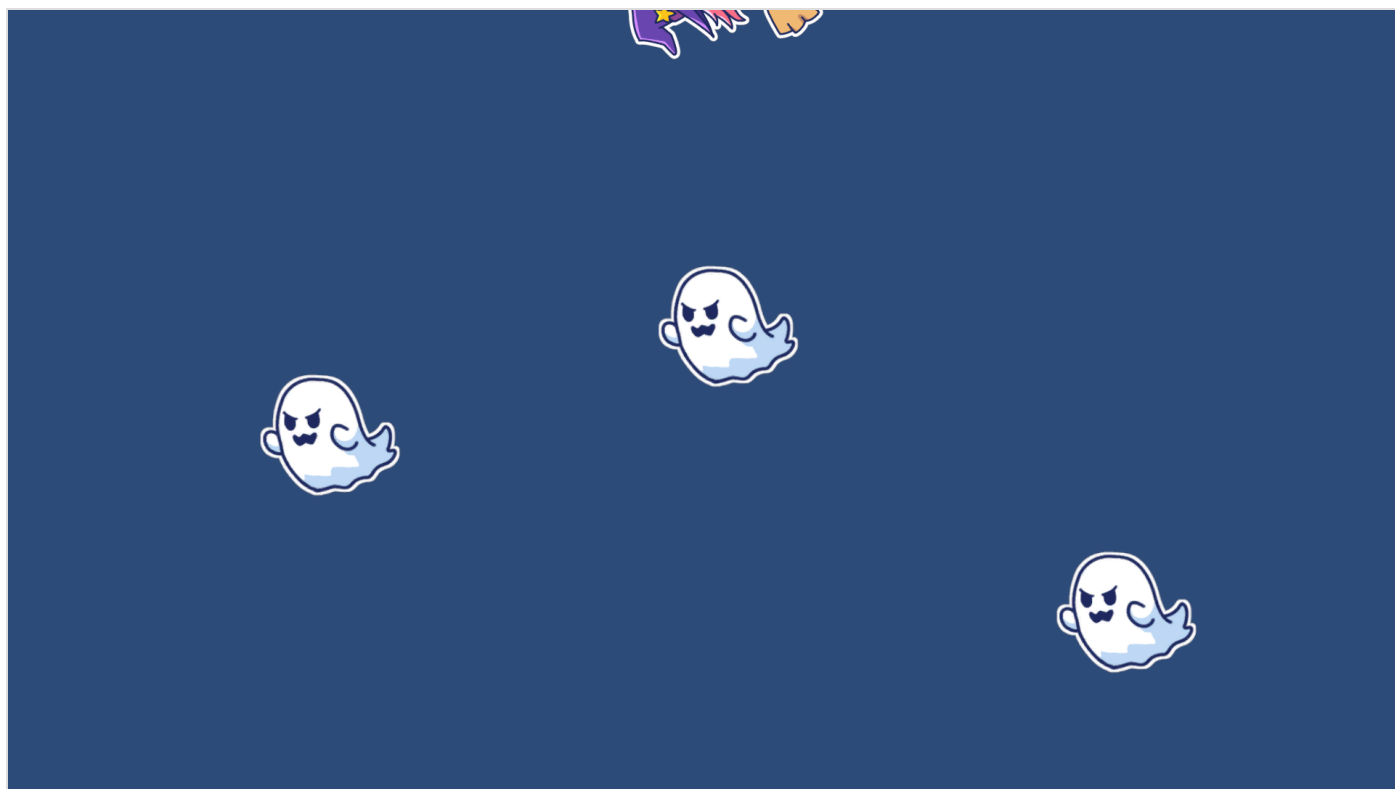
11. では、MagicalShooting.exe（Macの人は.app）をダブルクリック！

Unityエディターを介さず、ゲームが単体で起動したら大成功です！ 自分のゲームが「製品」になった瞬間です。

※Macの人は「開発元が未確認のため開けません」と出ることがあります。その場合は.appを右クリック→「開く」で起動できます。



もちろんゲームループもまるごと動きます。フルスクリーンで遊ぶ自分のゲームは感慨深いですよ。



12. このフォルダをまるごとzipに圧縮して渡せば、友達のPCでも遊んでもらえます（Windows向けビルドはWindowsの人に、Mac向けはMacの人に）。
※初回起動時に「WindowsによってPCが保護されました」と青い画面が出ることがあります。「詳細情報」→「実行」で起動できます。知らない発行元のアプリに出る警告なので、慌てなくて大丈夫。
13. ゲームが単体で動いたら、今回は完成です！
次回ははいよいよ最終回、Web向けにビルドしてunityroomに投稿します！
14. ↓ ---- **質問コーナー** ---- ↓

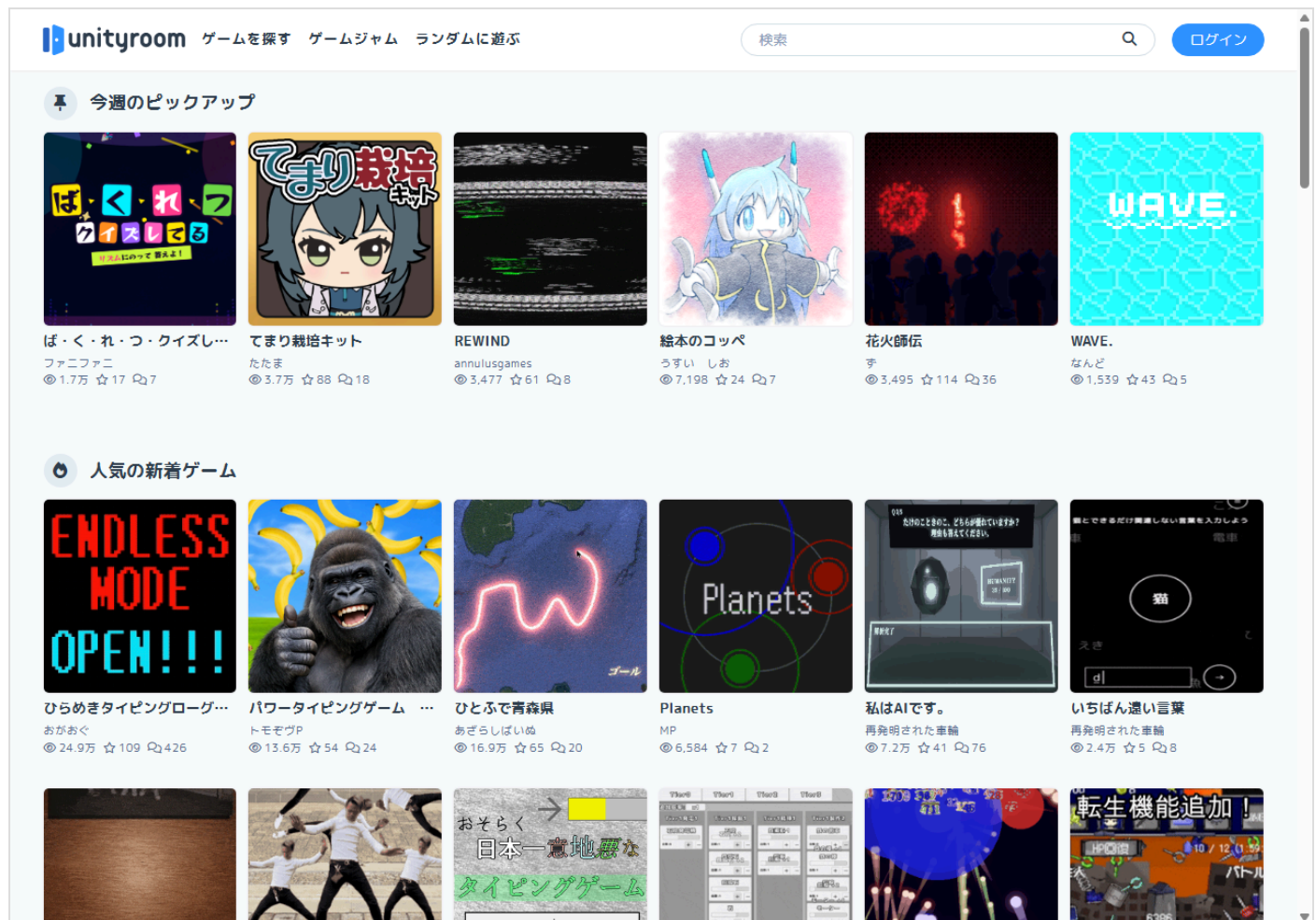
【5-05】UnityRoomに投稿しよう！【5章：ゲームループ】

1. クリア条件：Web向けにビルドして、unityroomに投稿し、ブラウザで遊べる状態にする
2. ついに最終回です。今回は自分のPC向けに書き出しましたが、今回はWeb（ブラウザ）向けに書き出して、世界に公開しましょう。

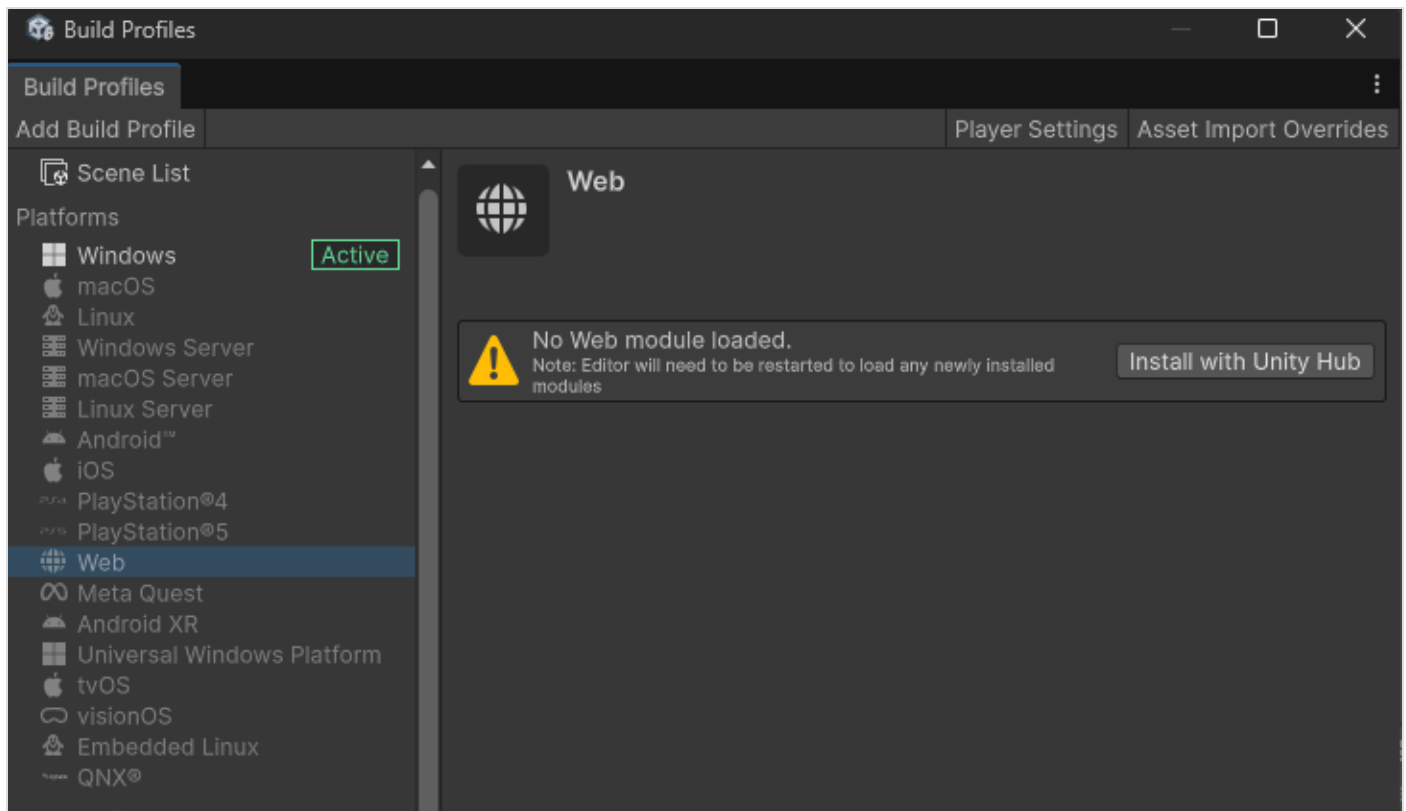
投稿先はunityroom。無料でUnity製ゲームを投稿・プレイできる、日本最大級のサイトです。

<https://unityroom.com/>

眺めているだけでも「これ、自分が作ったのと同じ仕組みで動いてるのか」と思えるゲームがたくさんあるので、遊んでみてください。



3. まずはWeb向けのビルドの準備から。File > Build Profiles を開いて、Platformsから「Web」を選択してください。
4. このとき、「No Web module loaded.」と表示された人もいます。

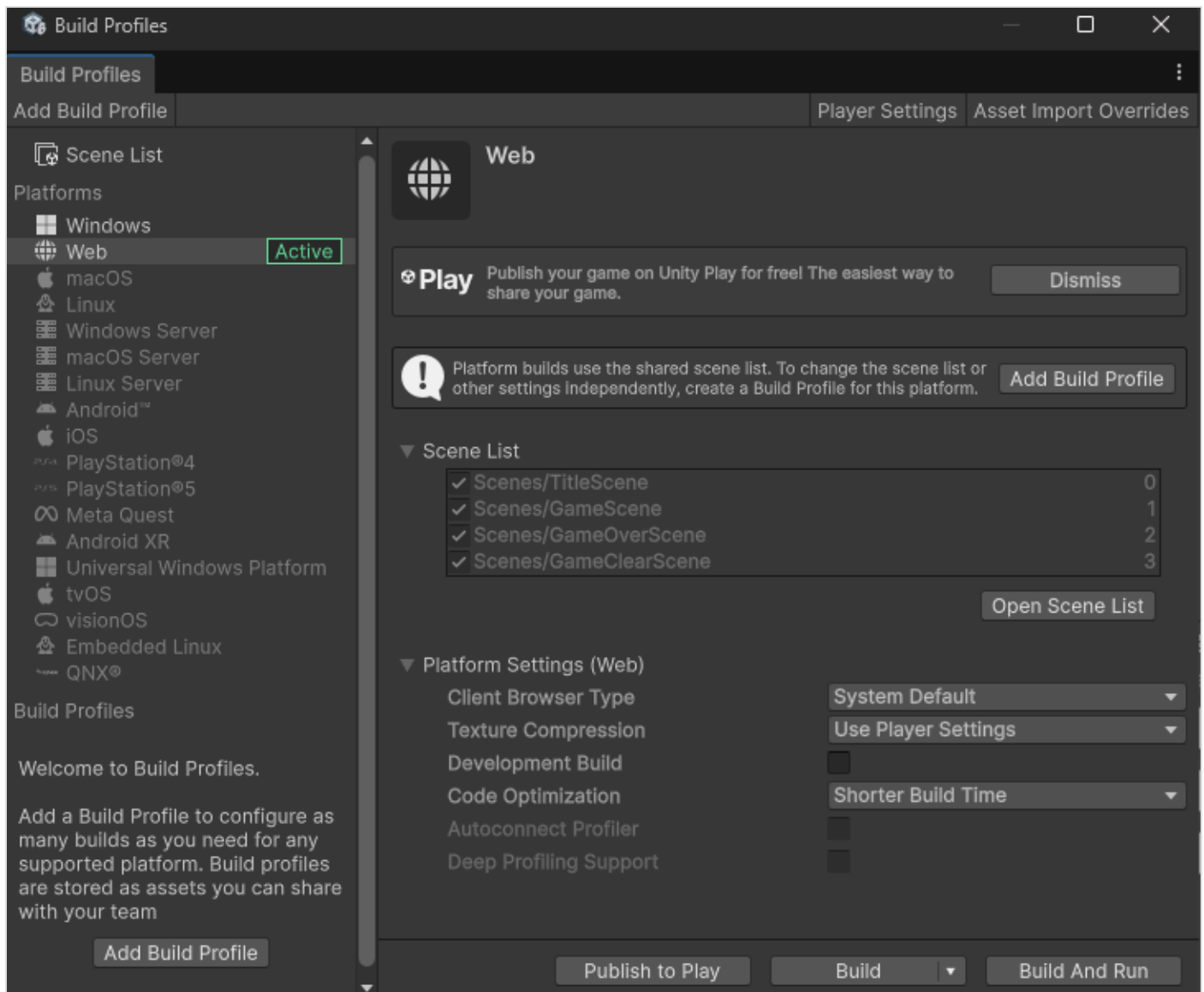


これはWeb Build Support（Web向けに書き出す部品）が入っていない状態です。ちなみに僕も入っていませんでした。「Install with Unity Hub」を押すか、1-03を見ながらUnityHubのInstallsからWeb Build Supportを追加してください。
※インストール後はUnityの再起動が必要です。再起動したらまたBuild Profilesに戻ってきましょう。

5. Webを選択して「Switch Platform」を押しましょう。

Unityが素材をWeb用に変換し直すので、数分かかります。終わるとWebに緑の「Active」が付きます。

※WindowsでもMacでも、ここから先の手順は同じです。



6. 次に、ビルドの設定を2か所いじります。Build Profiles右上の「Player Settings」を開いてください。
 (Edit > Project Settings > Player でも同じ画面が開きます)

7. 1か所目。Resolution and Presentation を開いて、

- Default Canvas Width : 960
- Default Canvas Height : 540

に設定しましょう。unityroomの標準サイズです。今まで作ってきた16:9の画面比率とも一致しています。1-05で16:9にしておいたのはこのためでした。



8. 2か所目。Publishing Settings を開いて、

- Compression Format : Gzip
- Decompression Fallback : チェックが入っていたら外す

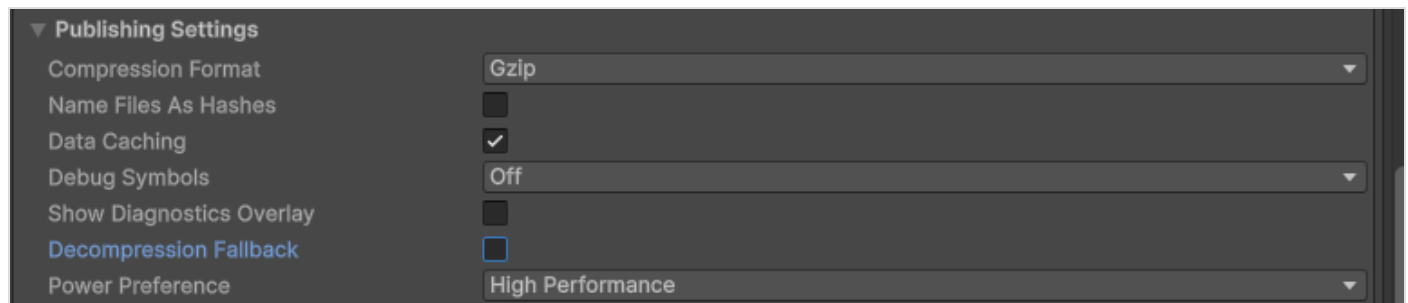
に設定しましょう。

用語 : Gzip

ファイルを小さく圧縮する形式。zipの親戚です。ゲーム本体はけっこう大きいので、圧縮しないと遊ぶ人のロード時間が長くなってしまいます。ブラウザはGzipを標準で解凍できるので、遊ぶ側は何も意識しません。

Decompression Fallbackは「解凍できない環境向けに、解凍プログラムをゲームに同梱する」保険のような設定ですが、今のunityroomでは不要どころか、公式が「チェックを外す」を指定しています。

この設定を間違えると、unityroomにアップロードしてもゲームが動きません！ 要注意！



9. 設定ができれば、Build Profilesに戻って「Build」ボタンを押しましょう。

前回と同じように書き出し先を聞かれるので、今度は「Web」用のフォルダを作って選択してください。

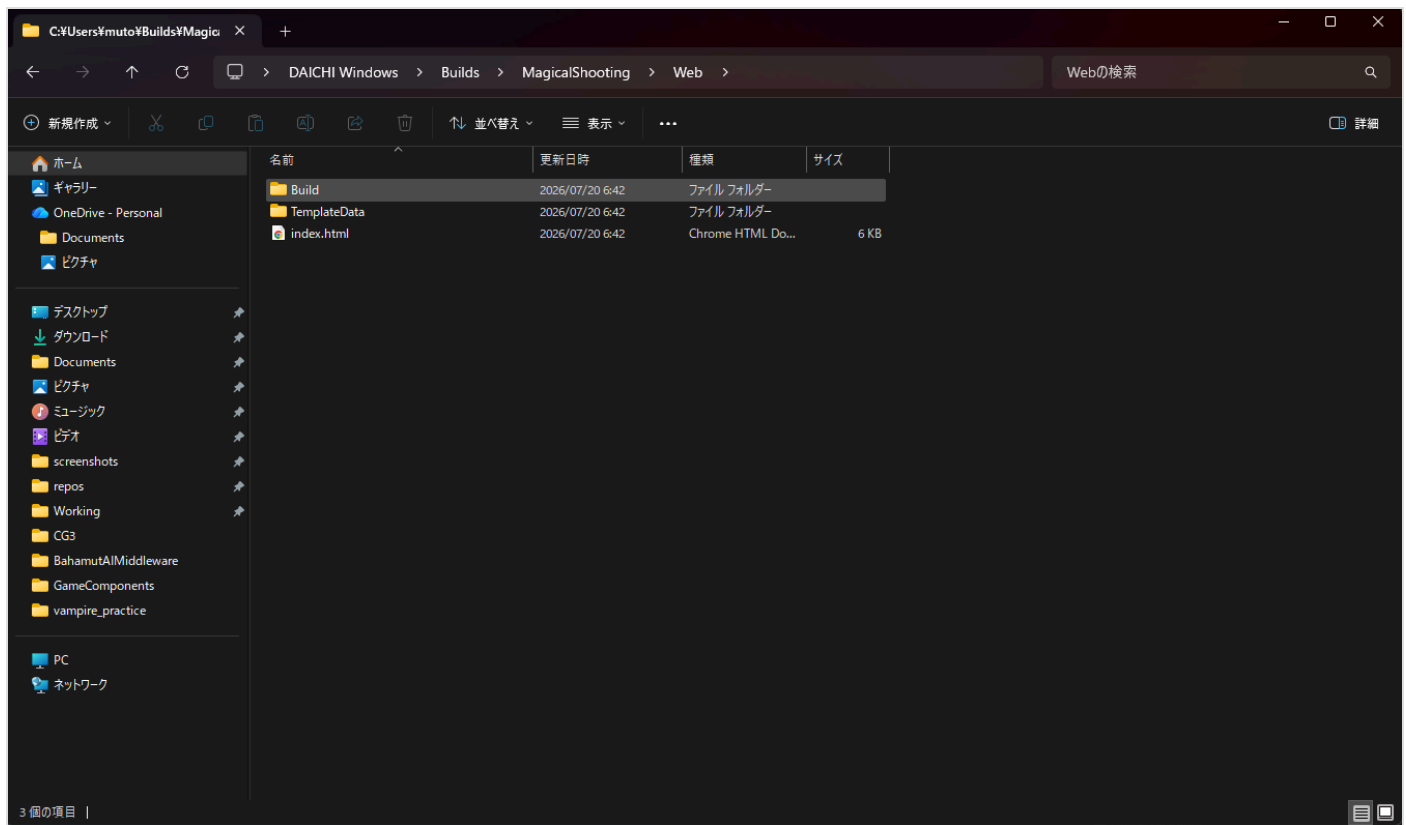
僕は「C:¥Users¥ユーザー名¥Builds¥MagicalShooting¥Web」にしました。

10. Webビルドは、PC向けよりだいぶ時間がかかります。初回は10分以上かかることも。

ブラウザで動く形式に全部変換し直しているので、気長に待ちましょう。

11. ビルドが終わると、フォルダの中にこんなものができています。

- index.html
- Buildフォルダ
- TemplateDataフォルダ



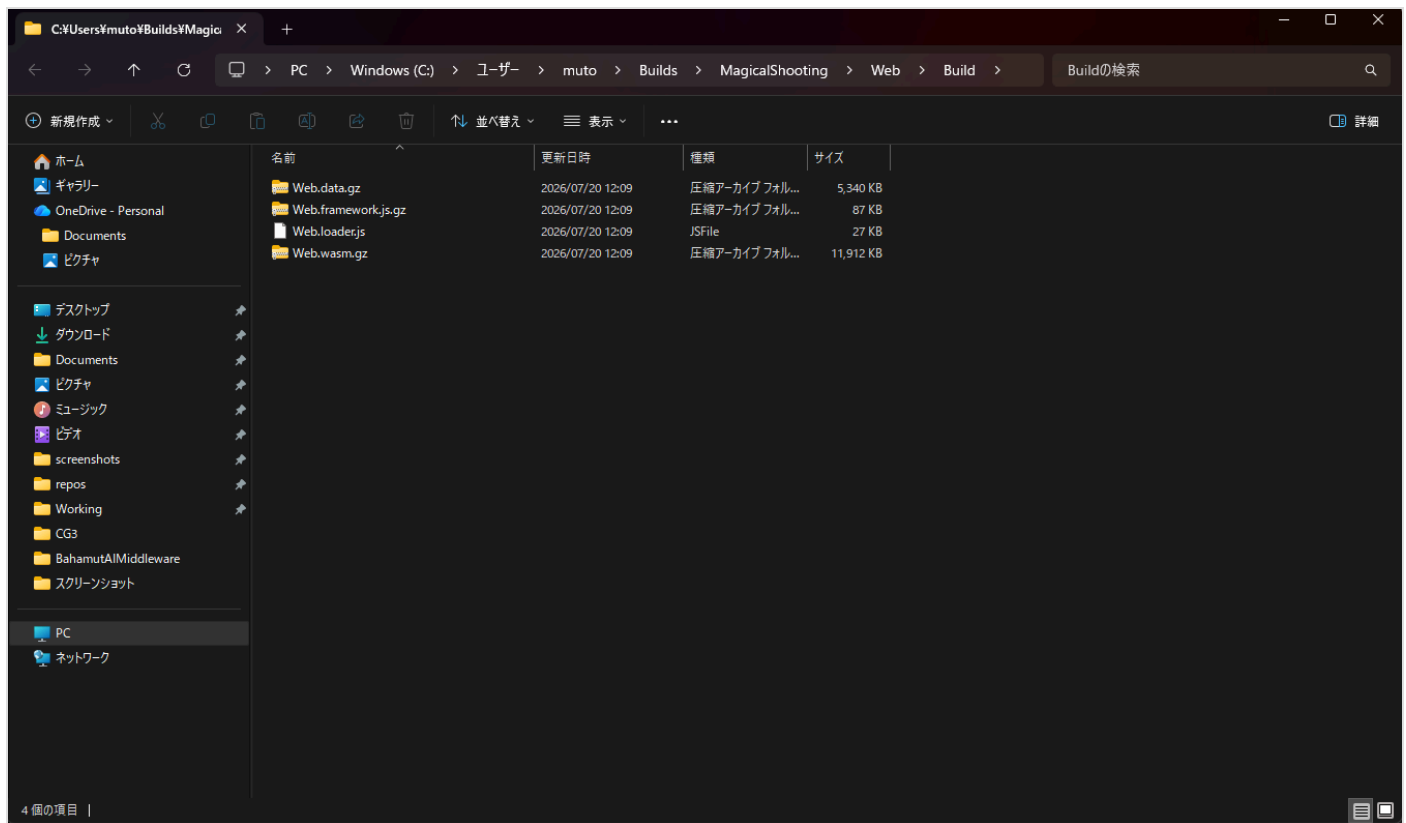
さらにBuildフォルダの中を見ると、4つのファイルがあります。

- .loader.js
- .data.gz
- .framework.js.gz
- .wasm.gz

この4つが入った「Buildフォルダ」を、あとでunityroomにアップロードします。場所を覚えておいてください。

※ファイル名の「○○」部分は、書き出し先のフォルダ名になります。僕の場合は「Web」フォルダに書き出したので「Web.loader.js」などになっていました。

※拡張子が「.gz」ではなく「.unityweb」になっている人は、Decompression Fallbackのチェックが外れていません。外してもう一度ビルドしましょう。



ちなみに「index.htmlをダブルクリックすれば遊べるのでは？」と思った人、鋭い。しかし残念、ブラウザのセキュリティ制限で、手元のファイルから直接は起動できません。動作確認はunityroomにアップしてから！

12. ここからはunityroomでの作業です。まずはアカウント作成。unityroomのトップページ右上の「ログイン」からユーザー登録をしましょう。

13. ログインしたら、「ゲーム投稿」からゲームの基本情報を入力します。

- タイトル：ゲームの名前。「Magical Shooting」でも、自分で考えた名前でもOK！
- ゲームID：ゲームページのURLの一部になります。半角英数で。あとから変更できないので慎重に！
- 紹介文：どんなゲームか。「魔女っ子が魔法弾で敵を撃ちまくるシューティング！」とか
- 操作方法：矢印キーで移動、スペースキーでショット。書いておかないと誰も操作がわかりません

 スクショ：ゲーム投稿フォーム（ログイン後に撮影）

14. 次に、WebGL設定でUnityのバージョン（6000.3.00）を選択し、画面サイズを960×540に設定しましょう。さっきビルド設定した数字と同じです。

15. いよいよアップロードです。「Webビルドアップロード」の画面に、書き出した「Build」フォルダを、フォルダごとドラッグ＆ドロップしましょう。

中の4つのファイルが自動でアップロードされて、「ファイルアップロード状況」が全部「完了」になればOKです。

※アップロード画面にも「Compression Format：Gzip」「Decompression Fallbackのチェックを外す」と注意書きがあります。さっきUnityで設定したやつですね。ここでエラーが出た人は、設定を見直してもう一度ビルドしましょう。

 スクショ：WebGLアップロード画面（ログイン後に撮影）

16. アイコン（サムネイル画像）も設定しましょう。

ゲーム画面のスクショでOKです。Gameビューを撮って設定しましょう。一覧ページで最初に見られるのはサムネなので、ゲームの顔だと思って選んでください。

17. 公開設定は、最初は「限定公開」がおすすめです。

限定公開はURLを知っている人だけが遊べる状態。まず自分でこっそり動作確認してから、堂々と公開しましょう。

18. 保存したら、自分のゲームページを開いて動作確認です！

- タイトルが表示されるか
- スペースでゲームが始まるか
- 敵を倒せるか、やられたらゲームオーバーになるか
- 20体倒したらゲームクリアになるか
- ちゃんとタイトルに戻ってくるか

最初の読み込みには少し時間がかかります。画面が変わらなくても、慌てずに待ってください。

19. 動作確認ができれば、公開設定を「公開」に変更！これであなたも、ゲームを世に出したゲーム開発者です。

 スクショ：公開されたゲームページ（公開後に撮影）

20. 完成！！そしてこれで、全チュートリアル完了です！！

Unityのインストールから始めて、キャラを動かし、弾を撃ち、敵を出し、ゲームループを組んで、公開までたどり着きました。本当にお疲れ様でした。

ゲームのURLは、ぜひこのスレに貼って自慢してください。全員分遊びに行きます。

21. ここから先は自由です。敵の種類を増やすもよし、スコアを付けるもよし、BGMを鳴らすもよし。

「こうしたら面白いかも」を試しては壊すのが、ゲーム開発の一番楽しいところです。良い開発ライフを！

22. ↓ --- **質問コーナー** --- ↓